

目次

1. はじめに
2. プレイヤーを表示
3. プレイヤーを動かす
4. マップの属性を読み取る
5. プレイヤーと敵を共通化する
6. BumpAction
7. スケルトンのAIを作る
8. 複数のスケルトンの追加
9. 戦闘システム
10. ヘルスバーを追加
11. アイテムを追加
12. インベントリUIを表示
13. アイテムを使う
14. 飛び道具を作成
15. FoV(Field of View)を追加
16. レベル2を作成
17. シーンマネージャー
18. 勝利条件を追加
19. デバック
20. おわりに

chapter01

はじめに

本書は、伝統的ローグライクゲームに興味があってしかも作ってみたい思う**奇抜な方**に向けてその実装のさわりを体験してもらえるように作成しました。ゲーム制作のチュートリアルは優れたものがたくさんありますが、特定のシーンだけ実装したものが多いように感じます。本書ではタイトルやエンディングなども含めてそれらをすべてのつながりを作ってパッケージにするところまでを扱います。（あまりないなら作ってみようと考えて取り組んだのが本書ですが、作業を終えた今、なかった理由がわかった気がします）本書が同じように考えている人の一助になれば幸いです。

01 注意書き

使用する環境はGodot4.5以降です。Godotに関して公式のチュートリアルなど何らかのものを終了したことがある方を対象としています。そのため、次のことについては詳しく触れていませんのでご注意ください。

Godot初心者に向けてはいますが、完全に初めてだとちょっと辛いです。

- Godotでスクリプトやシーン、フォルダなどを新規作成する操作
- シーンツリーにノードを追加する操作
- プロジェクト設定でインпутマップを作成する操作
- プロジェクト設定でメインシーンを設定する操作
- インスペクタでSprite2DのtextureをTextureAtlasから設定する操作
- TileMapLayerでtextureを設定する操作
- ノードにスクリプトをアタッチする操作
- AnimationPlayerノードで新規アニメーションを作る操作
- GDscriptの文法についての解説
- この書籍でのGodotロゴ使用はGodot Foundationの商標ポリシーに基づいて許可されたものであり、Godot Foundationとの公式な提携・後援を意味するものではありません

02 改訂版

本書は2026年8月に改訂されました。

進める内容に関しては変わりませんが、クラス名や実装方法が変わっている部分があります。前の版で途中まで進めていた場合、整合性が保てなくなる箇所がありますのでご注意ください。

改定版のソースコードは

<https://github.com/enclot/tutrogue>

revised ブランチにあります。(現在は子のブランチがデフォルトになっています)

主な変更点は

- 移動が4方向になった
- 入力ハンドラの見直し
- カスタムシグナルの積極的な利用
- アクションの種類の見直し
- グループの種類の見直し
- メインループの導入
- アクションを実行するのはメインループだけ
- FoVの表示方法変更
- Actor関係のクラス構造の見直し
- UI関係の実装の見直し

などです。

ちょっと補足

ソースコードをPDFからコピペすると改行が入ってしまって、ただしくコピペできないことがあります。AcrobatReaderでPDFを開いたときにそうなります。ChromeでPDFを開いたときは正しくコピペできることを確認しました。

03 伝統的ローグライクとは

Rogue(1980年)を原典とするその派生ゲームのジャンルをローグライクといいます。ただ現在はさまざまなジャンルのものがローグライクと呼ばれ、何がローグライクなのかよくわかりませんが、本書で扱う伝統的ローグライクは、Rogue、ADOM (Ancient Domains of Mystery)、Angband、Crawl、Nethackのような初期のローグ派生ゲームを思い浮かべてください。

ベルリン解釈(2008年)によるとローグライクの重要な要素として以下のものがあげられるそうです。

- ランダムな環境生成
- 恒久的な死
- ターン制
- グリッドベース
- ノンモーダル
- 複雑さ
- リソース管理

など

- 恒久的な死
- ターン制
- グリッドベース
- ノンモーダル

これらの要素を含むゲームを作っていきましょう。

残念ながら、一番重要なランダムな環境生成に関しては本書では扱いません。

https://www.roguebasin.com/index.php?title=Berlin_Interpretation

04 Tutorial Tuesday

アメリカ発の掲示板SNSのRedditに [r/roguelikedev](https://www.reddit.com/r/roguelikedev/) というコミュニティがあります。

<https://www.reddit.com/r/roguelikedev/>

このなかで、Tutorial Tuesdayというスレッドがあります。もとのチュートリアル環境はpythonを使ったものですが、参加者は、様々な開発環境でそのチュートリアルにチャレンジして進捗を共有していく趣旨です。チュートリアル自体が難しいので、完走できる参加者は少ないのですが、参加者たちの熱意を感じます。

スレッド

https://www.reddit.com/r/roguelikedev/comments/1nd215w/roguelikedev_tutorial_tuesday_2025_a_summary/

過去の参加者

https://old.reddit.com/r/roguelikedev/wiki/python_tutorial_series

本書はTutorial Tuesdayを大いに参考にしました。一度チュートリアルをこなしてみたら、再度、実装方法や実装項目を追加したり削ったり順番を変えたりしながら、自分に腹落ちできるような形に落とし込んでいったのが本書になります。

05 Godot命名規則

なるべく、以下のGDScriptスタイルガイドに従うようにしています。

https://docs.godotengine.org/ja/4.x/tutorials/scripting/gdscript/gdscript_styleguide.html

要素	記法
クラス名	UpperPascalCase
ノード名	UpperPascalCase
ファイル名	lower_snake_case
メソッド名	lower_snake_case
プロパティ	lower_snake_case
プライベート	先頭に_をつける

ちょっと補足

ソースコードが横に長くなってしまったときに、紙面の都合上はみ出ることがあります。

GDScriptの仕様では自由に改行できないケースもあるので、その場合は、`\` (バックスラッシュ)が追加してあります。

バックスラッシュがある行は、改行しても継続していることになります。

ソースコードを入力するときは適宜バックスラッシュを消して1行で入力した方が見栄えが良いと思います。

メソッドの引数が横に長くなっている時は、カンマで改行を入れてあります。

これも同じように、改行を消した方がソースコードの見通しがよくなると思います。

本書のソースコードは以下からダウンロードしてください。

<https://github.com/enclot/tutroque>

chapter02

プレイヤーを表示

01 アセットの準備

ゲーム内で使用する画像やフォントを準備しましょう。

アセットのファイルは `res://` の下に `asset` フォルダを作成しその中にコピーしてください。



本書で使用するアセットは以下を使用しました。

<https://kenney.nl/assets/1-bit-pack>

<https://kenney.nl/assets/kenney-fonts>

ダウンロードしたファイルの中から、

`monochrome-transparent_packed.png`

`colored_packed.png`

`Kenney Pixel.ttf`

以上の3つのファイルを `asset` フォルダの中にコピーしておいてください。

02 レベルシーンを作成

ゲームのメインのシーンです。マップやプレイヤー、敵、アイテムなどが子として追加されます。まずは地下1階をエディター上で作成していきましょう。

Node2Dを基にノードを新規作成して、Level1という名前で保存してください。

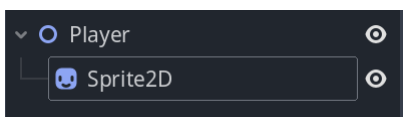
1. Node2D シーンを作成
2. 名前を Level1 に変更
3. 保存する level_1.tscn



03 プレイヤーシーンを作成

プレイヤーのシーンです。Node2Dを基にノードを新規作成して、Playerという名前で保存してください。それからPlayerノードの子にSprite2Dを追加してください。

1. Node2Dシーンを作成
2. 名前をPlayerに変更
3. Sprite2Dを子ノードとして追加



4. 追加したSprite2DのインスペクタのTextureを新規AtlasTextureにする
5. AtlasTextureをクリックしてAtlasのクイックロードから先ほどassetフォルダに追加したmonochrome-transparent_packed.pngを選ぶ
6. 領域を編集ボタンを押して、プレイヤーの画像の領域を決定する



7. インスペクタのOffsetのCenteredのチェックを外す

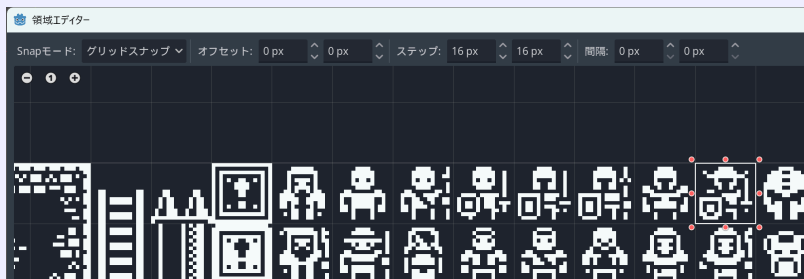
原点が左上になるようにします



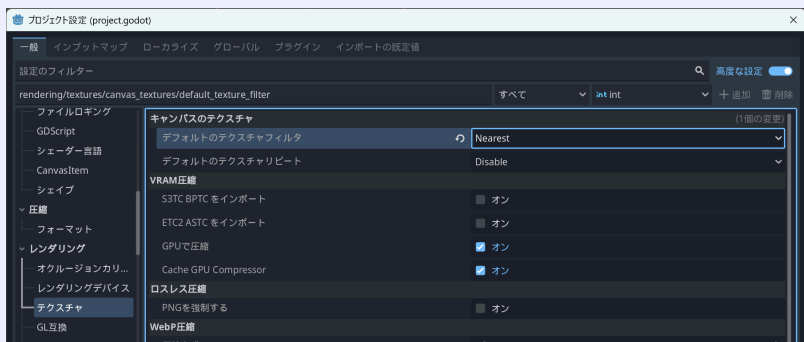
8. 保存する player.tscn

ちょっと補足

textureの領域を選択するとき、グリッドスナップにして、ステップを縦横16pxにして、選択する部分をドラックすれば簡単に選択することができます。



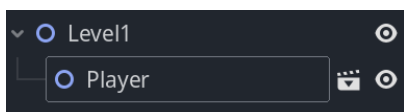
ドット絵のような低解像度のアセットを初めて読み込んで表示したとき、読み込んだ画像がぼやけているかもしれません。これはGodotが画素を補完しているせいで、メニューのプロジェクト-プロジェクト設定を開いて、レンダリング-テクスチャの中のデフォルトのテクスチャフィルタをNearestに変更してください。



04 プレイヤーシーンを インスタンス化して追加

先ほど作ったPlayerシーンをLevel1シーンへ追加します。

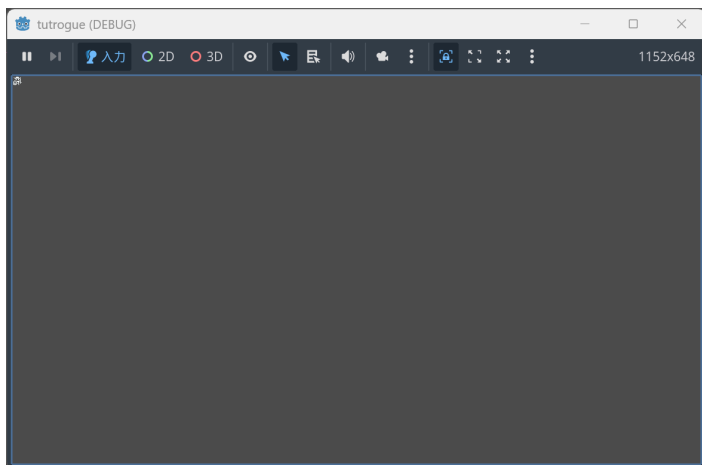
1. Level1 ノードの子にplayer.tscnをインスタンス化して追加



2. 動作確認してみる

一度動作確認をしてみましょう。

プロジェクトを実行します。始めて実行するときはメインシーンを選択する必要があります。level_1.tscnを選んでください。

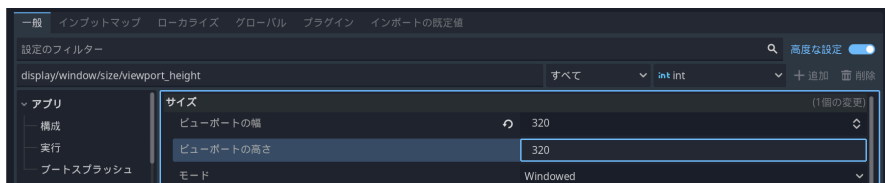


左上にめちゃくちゃ小さくプレイヤーが表示されていますね。プレイヤーのサイズが16x16なので画面に対してとても小さいです。

05 ゲーム画面のサイズ指定

タイルの大きさが16x16なので、20タイルぐらい表示できるように320x320の画面サイズにしましょう。

プロジェクト設定を開いてから、表示-ウィンドウからサイズ-ビューポートの幅、高さをそれぞれ320にします



ちょっと補足

現在のディスプレイの多くはフルHD(1920x1080)以上の場合が多く、このままのサイズでは小さくて見づらいです。プロジェクト設定を開いてから、表示-ウィンドウからサイズ-ウィンドウの広さ（高さ）のオーバーライドを1280に指定します。また、少し下のストレッチの項目のモードをviewportへ変更します。こうすることで、引き伸ばされて表示されます。



chapter03

プレイヤーを動かす

01 インпутマップを設定

キーボードの入力でプレイヤーを動かしましょう。斜め移動はちょっと直感的ではないですがよくあるいつものやつです。

1. プロジェクト設定から、以下のようなインพุットマップを設定

アクション	キー
move_up	矢印キー 上
move_down	矢印キー 下
move_left	矢印キー 左
move_right	矢印キー 右



02 ゲームデザインパターン

Commandパターン

ゲームにはそのゲーム内のアクション（移動、攻撃、アイテムを使うなど）があり、たいていはユーザーの入力によってそれらが実行されます。このゲーム内のアクションを分離してクラスとして定義するのが、Commandパターンです。

より詳しく調べたい場合は下記を参考にしてください。

<https://gameprogrammingpatterns.com/command.html>

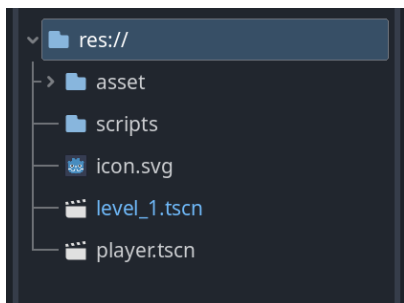
よくわからなくても実装は簡単なのでとりあえず作ってみると、何となく理解できるでしょう。

デザインパターンの名前はCommandなのですが、本書では基底クラスをActionとしています。

ActionはCommandと同じ意味として使っています。

これから、MovementAction(移動)やMeleeAction(近接攻撃)などを作っていきます。

03 スクリプトの保存場所



res://の下に**scripts**フォルダを作成します。

必須の手順ではありませんが、ファイルがたくさんになるとプロジェクト内がごちゃごちゃになりますので整理のためにスクリプトだけは別のフォルダ内に収めるようにしておきます。

04 入力を処理する

ユーザー入力を扱う `GameInputHandler` ノードを作成しましょう。

`_input(event: InputEvent)` を利用してキー入力があったときに処理を行います。インプットマップに定義した名前をキーとして向きの座標値の `Vector2i` が `DIRECTIONS` に定義されています。`_input()` の処理で方向キーの入力に対応した座標値が出力されます。

また、カスタムシグナルの `action_selected(action)` を定義して、キー入力があったときにアクションを作成してカスタムシグナルを発火させます。

1. スクリプトを作成する `game_input_handler.gd`

`game_input_handler.gd`

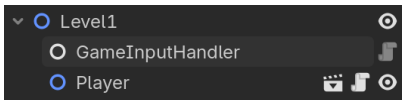
```
class_name GameInputHandler
extends Node

const DIRECTIONS = {
    &"move_up": Vector2i.UP,
    &"move_down": Vector2i.DOWN,
    &"move_left": Vector2i.LEFT,
    &"move_right": Vector2i.RIGHT,
}

signal action_selected(Action)

func _input(event: InputEvent) -> void:
    if event is InputEventKey and event.is_pressed():
        for action:StringName in DIRECTIONS.keys():
            if event.is_action_pressed(action):
                var offset: Vector2i = DIRECTIONS[action]
                print(action, " -> ", offset)
                # ここでアクションを作成してaction_selectedを発火させる
                break
```

2. Level1 ノードに子ノードとして GameInputHandler を追加



3. 動作確認してみる

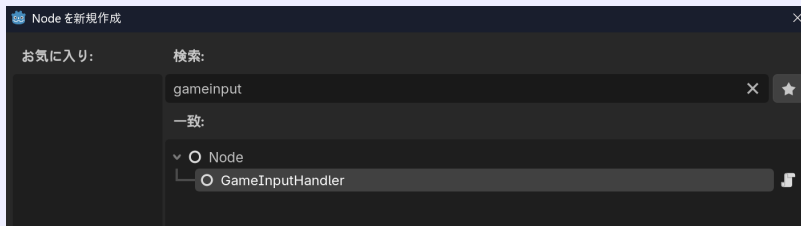
実行してみると方向キーに対応した座標値が表示されています。

```
Godot Engine v4.6.3.stable.official.7d41c59c4 - https://godotengine.org
OpenGL API 3.3.0 - Build 27.20.100.9664 - Compatibility - Using Device: Intel - Intel(R) UHD Graphics 620

move_right -> (1, 0)
move_left -> (-1, 0)
move_right -> (1, 0)
move_left -> (-1, 0)
move_up -> (0, -1)
move_down -> (0, 1)
```

ちょっと補足

MainGameInputHandlerはNodeを継承しているので、子ノードを追加するときに直接追加することができます。



05 MovementActionの作成

すべてのアクションの基底クラスになる Action を仮想クラスで作成します。

1. action.gd を新規作成

すべての Action は perform() にそのアクションを行いたいもの（ここでは Player）を引数で渡して、アクションの処理が実行されるようにしていきます。

action.gd

```
@abstract class_name Action extends RefCounted

@abstract func perform(_player:Player) -> bool
```

Action クラスを継承して MovementAction クラスを作成します。

MovementAction は移動量を引数にして初期化します。その移動量を使って perform() で移動の処理を行います。今は print() するだけになっています。また perform() はそのアクションが成功したかどうかの bool を返します。

2. movement_action.gd を新規作成

movement_action.gd

```
class_name MovementAction
extends Action

var offset:Vector2i #移動量

func _init(_offset:Vector2i) -> void:
    offset = _offset

func perform(_player:Player) -> bool:
    print("MovementAction perform ", offset)
    return true
```

06 GameInputHandlerでactionを作成する

2. game_input_handler.gdを変更

方向キーが入力されたときに MovementAction を作成してカスタムシグナルを発火(emit)させます。

game_input_handler.gd

```
class_name GameInputHandler
extends Node

const DIRECTIONS = {
    &"move_up": Vector2i.UP,
    &"move_down": Vector2i.DOWN,
    &"move_left": Vector2i.LEFT,
    &"move_right": Vector2i.RIGHT,
}

signal action_selected(Action)

func _input(event: InputEvent) -> void:
    if event is InputEventKey and event.is_pressed():
        for action:StringName in DIRECTIONS.keys():
            if event.is_action_pressed(action):
                var offset: Vector2i = DIRECTIONS[action]
                print(action, " -> ", offset)
                # ここでアクションを作成してaction_selectedを発火させる
                action_selected.emit(MovementAction.new(offset))
                break
```

07 プレイヤーがアクションを待つようにする

1. player.tscnを開いてPlayerノードにスクリプトをアタッチ

GameInputHandler のカスタムシグナルの action_selected を await する get_action()を作ります。

player.gd

```
class_name Player
extends Node2D

var input_handler:GameInputHandler

func get_action() -> Action:
    return await input_handler.action_selected
```

2. level_1.tscnを開いてLevel1ノードにスクリプトをアタッチ

シーンツリーのGameInputHandlerノードの参照をPlayerに渡します。また、無限ループでplayerのget_action()で入力を待ち続けます。シグナルが来たらactionのperform()を実行してループの初めに戻ります。

level_1.gd

```
class_name Level1
extends Node2D

@onready var player: Player = $Player
@onready var input_handler: GameInputHandler = $GameInputHandler

func _ready() -> void:
    player.input_handler = input_handler

    while true:
        var action: Action = await player.get_action()
        action.perform(player)
```

3. 動作確認してみる

実行して方向キーを押すと、actionがperformされていることがわかります。

```
move_up -> (0, -1)
MovementAction perform (0, -1)
move_down -> (0, 1)
MovementAction perform (0, 1)
move_left -> (-1, 0)
MovementAction perform (-1, 0)
move_right -> (1, 0)
MovementAction perform (1, 0)
```

08 Playerが動く処理

プレイヤーの表示が動くようにしましょう。

1. player.gdを変更

グリッド移動の場合、グリッド上の座標値と描画に使われる座標値が違うので、これを簡単に扱えるようにするために、セッターを使って grid_position に値が変更された時に同時に描画位置が変わるようにプロパティの position にも適切な値が代入されるようにします。また、移動量を現在の grid_position に足す move() を作成します。

player.gd

```
class_name Player
extends Node2D

var input_handler:GameInputHandler

var tile_size := Vector2i(16,16)

var grid_position:Vector2i:
    set(value):
        grid_position = value
        position = grid_position*tile_size

func get_action() -> Action:
    return await input_handler.action_selected

func move(_offset:Vector2i) -> bool:
    grid_position += _offset
    return true
```

2. movement_action.gdを変更

perform()の中でPlayerのmove()を呼び出します。

movement_action.gd

```
class_name MovementAction
extends Action

var offset:Vector2i #移動量

func _init(_offset:Vector2i) -> void:
    offset = _offset

func perform(_player:Player) -> bool:
    print("MovementAction perform ", offset)
    return _player.move(offset)
```

3. 動作確認してみる

入力によってプレイヤーの表示が移動するようになりました。

chapter04

マップの属性を読み取る

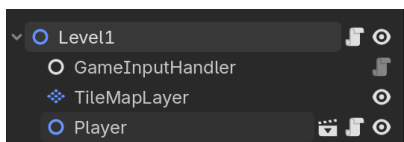
01 TileMapLayerの準備

ローグライクといえばランダム生成のマップですが、ランダムにマップ配置が変わってしまうと機能を作るときにデバックがしにくいことがあります。そこでTileMapLayerで地形配置できるようにしましょう。

1. TileMapLayerの追加

TileMap と TileMapLayer がありますが、TileMap は古くて非推奨なのでTileMapLayerを使ってください。

また、シーンツリーの下にある方が重なりの上に表示されるので、TileMapLayerをPlayerより上に配置します。

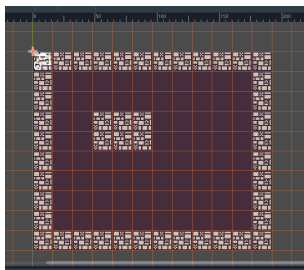


インスペクタのTile Setをクリックして新規作成します。

画面下のタブでTileSetを選んでTileSourcesの+ボタンを押し、アトラスを選択して**colored_packed.png**を選択します。

壁のタイルと床のタイルで適当にマップを作成します。

この時タイルが矩形に埋められるように配置するようにしてください。TileMapLayerのタイルが配置された矩形の中になにも配置していないグリッドがあると、後に実装するFOVの処理でエラーになります。



02 Playerの位置の調整

1. Playerのノードを選択して床のある位置へ配置

ちょっと補足

2D表示モードで、スナップの設定をして、グリッドのステップを16pxにしておくと簡単にピッタリ配置することができます。



2. 動作確認してみる

この状態で実行して移動してみると、最初の一步はワープしてしまいます。
これはPlayerの `grid_position` の初期値が(0,0)であるためです。
また、壁は突き抜けてしまいます。

03 プレイヤーの初期位置を取得

1. player.gd に追加

エディターで配置した位置で `grid_position` が得られるように `player.gd` に以下のスクリプトを追加します。

`_ready()` はシーンツリーに追加されたときに呼び出されますので、ゲームが開始して Level1 シーンが呼び出されその中で Player ノードがシーンツリーに配置されたときに `grid_position` に値が代入されます。

player.gd 一部

```
func _ready() -> void:
    grid_position = Vector2i(position)/tile_size
```

2. 動作確認してみる

最初に `grid_position` が代入されて、最初の一步でワープしなくなります。

Integer division. Decimal part will be discarded. の警告がでます。これは整数のデータ同士で割り算をしていて、小数点以下がすべて消えてしまいますが大丈夫ですか？という確認です。ここでは無視して構いませんが、気になるようなら以下のようにしてください。

player.gd 一部

```
func _ready() -> void:
    grid_position = Vector2i(position / Vector2(tile_size))
```

または

player.gd 一部

```
func _ready() -> void:
    @warning_ignore("integer_division")
    grid_position = Vector2i(position)/tile_size
```

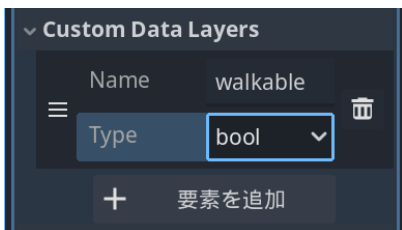
04 壁の当たり判定を追加する準備

TileMapLayerでは物理演算を使った当たり判定を使うこともできますが、グリッドベースの移動の場合は物理演算を使うまでもなく座標の比較でそのタイルが歩行可能かどうかを判定することができます。

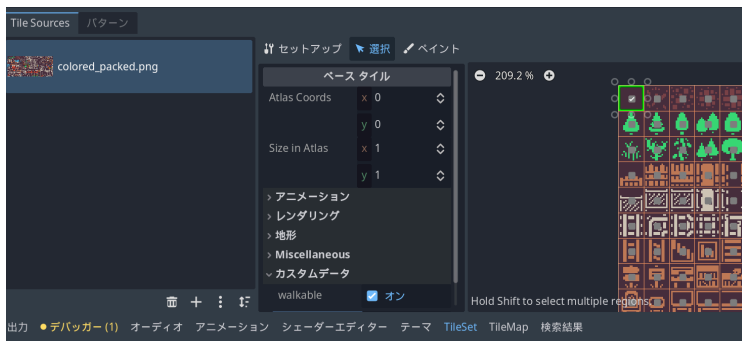
TileMapLayerのそれぞれのタイルに Custom Data として walkable(歩けるかどうか)を追加して歩ける場所かどうかを判断しましょう。

1. TileMapLayerの Custom Data Layerに以下の要素を追加

Name	Type
walkable	bool



シーンツリーでTileMapLayer ノードを選択し、画面下部のTileSetタブで、選択を選んで床のタイルを選択します それからカスタムデータの walkable をオンにします。



05 カスタムレイヤーを読み取るための準備

1. cell_property.gdを新規作成

先ほど TilemapLayer のカスタムレイヤーに設定した値を読み取るために、セルの属性を表す CellProperty を作成します。

cell_property.gd

```
class_name CellProperty
extends RefCounted

var is_walkable:bool = false

func _init( _walkable:bool) -> void:
    is_walkable = _walkable
```

2. cell_properties_map.gdを新規作成

セルの属性の集まりの CellPropertiesMap を作成します。 TilemapLayer からカスタムレイヤーを読み取ってすべてのセルの属性を管理します。 指定した座標の CellProperty を取得できる get_cell() と、その座標が範囲内かを調べる is_in_bounds() があります。

cell_properties_map.gd

```
class_name CellPropertiesMap
extends RefCounted

# セルの属性の集まり
var width:int
var height:int

var cells:Array[CellProperty]

func _init(_tilemap:TileMapLayer) -> void:
    var used_rect:Rect2i = _tilemap.get_used_rect()
    width = used_rect.size.x
    height = used_rect.size.y

    for y in height:
        for x in width:
            var data = _tilemap.get_cell_tile_data(Vector2i(x,y))
            var property = CellProperty.new(
                data.get_custom_data("walkable")
            )
            cells.append(property)

func get_cell(_pos:Vector2i) ->CellProperty:
    return cells[_pos.x + width*_pos.y]

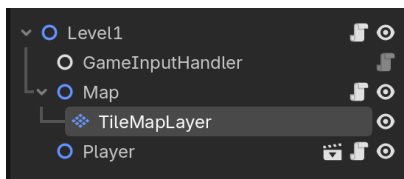
func is_in_bounds(_pos: Vector2i) -> bool:
    return (
        0 <= _pos.x
        and _pos.x < width
        and 0 <= _pos.y
        and _pos.y < height
    )
```

06 Mapノードの作成

Mapを管理するMapノードを作成します。

TilemapLayerをMapの子に移動します。

1. Level1 ノードに子ノードとしてNode2Dを追加
2. 名前をMapに変更
3. TilemapLayerをMapの子に移動



ちょっと補足

自分でもときどき混乱してしましますが、名前にcellとtileがあります。どちらもほとんど同じものですが、以下のような違いで使い分けようとしています。

Name	Type
cell	それぞれの格子 マス
tile	マスを描画する画像 主に表示に関係するときに
map	セルの集まり

07 TilemapLayerから カスタムレイヤーを読み取る

1. map.gdを新規作成してMapノードにアタッチ

`_ready()`でノード生成時に子のTilemapLayerから`cell_properties_map`を作成します。

`is_walkable()`で指定した座標が範囲内かどうか`walkable`かどうかを調べています。

map.gd

```
class_name Map
extends Node2D

var cell_properties_map:CellPropertiesMap

@onready var tilemap: TileMapLayer = $TileMapLayer

func _ready() -> void:
    cell_properties_map = CellPropertiesMap.new(tilemap)

func is_walkable(_pos:Vector2i) -> bool:
    return cell_properties_map.is_in_bounds(_pos) and \
        cell_properties_map.get_cell(_pos).is_walkable
```

08 プレイヤーの移動処理を変更

Mapへの参照を持ちます。移動先がwalkableでなかったら false を返してキャンセルします。

1. player.gd を変更

player.gd

```
var map:Map

func move(_offset:Vector2i) -> bool:
    var target_pos:Vector2i = grid_position + _offset
    if map.is_walkable(target_pos):
        grid_position = target_pos
        return true
    return false
```

2. Player に Map の参照を渡す

Player がシーンツリーに追加された後に player に map への参照を渡しておきます。

level_1.gd

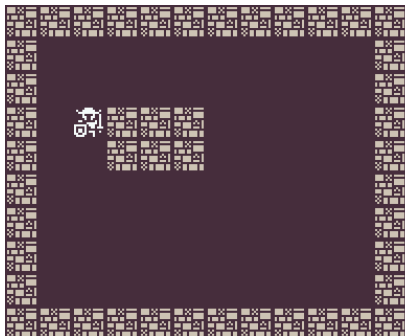
```
class_name Level1
extends Node2D

@onready var player: Player = $Player
@onready var input_handler: GameInputHandler = $GameInputHandler
@onready var map: Map = $Map

func _ready() -> void:
    player.input_handler = input_handler
    player.map = map
    while true:
        var action:Action = await player.get_action()
        action.perform(player)
```

3. 動作確認してみる

壁に向かって動いても突き抜けなくなっています。



chapter20

おわりに

おつかれさまでした。

一応のゲームとしての遷移の体裁ができたと思います。ただし、ゲームとして完成したかというところからが始まりになるでしょう。現状ではレベルも2つだけ敵は1種類でランダムな要素は全くありません。敵の種類を増やし、アイテムを増やし、戦闘システムを奥深いものにするなど途方もない作業が始まります。

システム面でも恐怖で逃げ出すなど敵のステータスによってAIを入れ替えたり、移動や攻撃にアニメーションをつけたりやることはまだまだたくさんあります。

今回利用したデザインパターンのCommandやComponentは、できることを増やすには最適な方法だと思います。すこしでも伝統的ローグライクをつくる助けになれたのであれば幸いです。

もしご好評いただければ、ランダムなレベル生成や敵のAIのバリエーション追加など、より深めた内容で続編を作りたいと考えております。

01 改訂版について

改訂版ではGodot4.6.3stableを使って確認しています。パフォーマンスの改善やエディタのQoLの改善が主なので本書にかかわる部分では、4.5とほとんど同じだと思います。

まさしく、ゲームがなかなか完成させられない人ムーブで、設計とかクラス名とかが気になってとりかかりました。前の版で十分に検討したつもりですが、なんでこんなふうにやってんのと自分にツッコミをいれたいくなるような中途半端な設計でちょっと恥ずかしくなりました。また、自分でこれを基に拡張しようとしたときに、作りづらかったところも見直しています。

改定後の拡張の内容としては、

- ターンエネルギー制の導入 Actorによって素早さがちがう。自分が1回動く間に敵が2回うごくとか
- ラइटニングシステム FoV以外に、照明などで明るさが変わる
- 攻撃予告エリア 一定ターン後にダメージを受ける呪文攻撃など

等の要素が追加できていますので、設計がすこしブラッシュアップされたかと思っています。（また、直したくなるかもしれませんが）

●謝辞

本書を制作するにあたって多くのインターネット上の知識やソフトウェアに助けられました。感謝申し上げます。

- Godot Engine

<https://godotengine.org/>

- Reddit r/roguelikedev

<https://www.reddit.com/r/roguelikedev/>

- ゲームデザインパターン

<https://gameprogrammingpatterns.com/>

- Youtube でチュートリアル動画

<https://www.youtube.com/@baconandgames>

<https://www.youtube.com/@MostlyMadProductions>

<https://www.youtube.com/@uheartbeast>

- アセット

<https://kenney.nl/>

<https://free-game-assets.itch.io/free-summer-pixel-art-backgrounds>

- Vivliostyle

<https://vivliostyle.org/>

- 伝統的ローグライク

https://www.youtube.com/@canary_yellow

●著者紹介

ゲームが完成させられないプログラマー歴20年以上。

好きなローグライクはZorbus、Mangui (만귀고개) です。

<https://www.zorbus.net/>

<https://store.steampowered.com/app/3220010/Mangui/>

●奥付

2025年12月1日 初版発行

2026年8月19日 改訂版発行

- 著者 遠藤孝則
- 連絡先 en_tk_m@yahoo.co.jp